Data Analytics Project: Railway Gauge Trends

This document outlines a structured, end-to-end data analysis of a project based on railway gauge data.

Dataset Used: railway_gauges_1.csv

Project Goal: To analyze historical trends and composition of different railway gauges (Broad, Metre, Narrow) over the years.

Modules Used: The primary libraries/modules required for this project are,

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (np.diff).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts).

Project Coverage

Category	Skills Covered
Data Pre-processing	Data Loading, Missing Value Handling, Type Conversion
Pandas Operations	Filtering, Selection, Aggregation, Feature Engineering
NumPy Calculations	Yearly growth calculation (np.diff)
Visualization	Line Graph, Bar Chart, Pie Chart, Stacked Bar Chart

Requirement of the projects are, Basic Data Loading & Cleaning, Simple Visualization, Filtering and Bar Chart, Feature Engineering and Pie Chart, Advanced Analysis and Multiple Graphs which must cover Data Preprocessing, Pandas Operations, NumPy Calculations and Visualization using Line graph, Bar chart, Pie chart and Stacked chart.

Scenario 1: Basic Data Loading & Cleaning

Task Description: In this task we are loading the dataset from local computer to dataframe. Once that is done we can check the top and last rows using head and tail functions. Then we need to check for any missing values in the provided dataset and if there is any, we have to replace that NaN with 0. And convert the selected columns into numeric type for further processing.

Task	Description
1	Load the dataset into a Pandas DataFrame.
2	Display the first 5 rows and column names.
3	Check for missing values and replace them with 0.
4	Convert all gauge columns (Broad, Metre, Narrow, Total) to numeric types.

Task Workflow:

The very first step is to create a proper directory structure having main.py file along with the dataset railway_gauges.csv in the same folder. If both are in different paths, change the path in the code as and when required.

Before typing code, we need to import pandas, numpy and pyplot to the main python file.

```
from matplotlib import pyplot as plt
import numpy as np
import pandas as pd
```

In the above code, matplotlib(pyplot is the sub-module of matplotlib), numpy and pandas are the module names which are required to perform operations as requested. plt, np and pd are the aliases of module names for using convention.

To load the given dataset to pandas DataFrames, we are using the code as shown below.

```
# Load the dataset into a Pandas DataFrame.
df=pd.read_csv('railway_gauges_1.csv')
```

In the above code,

df => variable name for storing/accessing DataFrame.

read_csv => panda function to read the data from CSV file.

In pandas, to view the top 5 rows from a dataset we can use head() function. We can use the head function as shown below.

```
# Display the first 5 rows and column names.
print(df.head())
```

The above code will give you the output of top 5 rows from the given dataset along with the column names along with the index values, which is 0 to 4(n-1) as shown below.

```
      Year  Broad Gauge  Metre Gauge  Narrow Gauge  Total
0  1964-65         3521         2891           464   6876
1  1965-66         3603         2913           470   6986
2  1966-67         3615         2911           481   7007
3  1967-68         3601         2920           506   7027
4  1968-69         3603         2921           508   7032
```

In pandas, to check if a DataFrame object have NaN(Not a Number) in any rows we can use isnull() function. Basically this function is used to check for empty or missing values from a table or dataset. It will check each cell and returns True if the cell is empty and False if the cell has some data.

Along with isnull() we can use sum() function to calculate the total number of cells having NaN value or empty cells.

If we have any missing values in the given dataset, we can replace those cells with 0(Zero) by using df.fillna() function as shown below.

```
# Checking missing values per column
print("Missing values are\n",df.isnull().sum(),'\n')
```

This code will check each column for missing values and will provide us the missing values along with the count as shown below.

```
Missing values are
 Year            0
Broad Gauge      0
Metre Gauge      0
Narrow Gauge     0
Total            0
```

Here year, Broad Gauge, Metre Gauge, Narrow Gauge and Total are the column names and each having no missing values.

If we need to check for rows we can use the same df.isnull() function along with axis=1. This will tell python to check horizontally in the given dataset and return the row where exactly the missing value is present.

```
# Returns only rows with NaN
missing_rows = df[df.isnull().any(axis=1)]
print(missing_rows)
```

This code will result in Empty DataFrame as there are no missing values in any row.

```
Empty DataFrame
Columns: [Year, Broad Gauge, Metre Gauge, Narrow Gauge, Total]
Index: []
```

Suppose if there are any missing values and we need to replace it with some other values, for example 0(Zero). We can use df.fillna() function. This function will check for NaN and replace the NaN with specified value. To achieve that we can use the following code.

```
#Replacing missing values with 0
df.fillna(0, inplace=True)
```

In the above code df.fillna() is the function name to replace the missing values. 0 within the parenthesis is the value which we want to replace with, inplace=True is the condition. As we run the above code it will replace all the missing values with 0 and stores in computer memory. If we want we can use fillna() function with single columns too.

Example: df['Broad Gauge'].fillna(0, inplace=True)

The next task is to convert particular columns to Numeric type. In the provided dataset we do have column names, so with the help of those column names we can change the values of each column to numeric type. To achieve that pandas provide an function called pd.to_numeric. This function accepts columns and change those columns into numeric type. If we have any blank spaces or –(hyphen) or hidden strings in the cells program may crash. To avoid that we can use errors='coerce'. This will prevent the code from crashing. And we can use fillna(0) function to replace NaN values with 0 to keep our data clean.

```
# List of columns to convert
cols = ['Broad Gauge', 'Metre Gauge', 'Narrow Gauge', 'Total']
# Convert them to numeric, turning errors like '-' or ' ' into NaN
df[cols] = df[cols].apply(pd.to_numeric, errors='coerce')
# Fill any new NaNs with 0 to keep the data clean
df[cols] = df[cols].fillna(0)
print(df.info())
```

df.info function is used to get all the details of an DataFrame like their class type, RangeIndex, total number of rows and columns, size etc.. as shown in the below picture.

```
<class 'pandas.DataFrame'>
RangeIndex: 49 entries, 0 to 48
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Year          49 non-null     str
 1   Broad Gauge   49 non-null     int64
 2   Metre Gauge   49 non-null     int64
 3   Narrow Gauge  49 non-null     int64
 4   Total         49 non-null     int64
dtypes: int64(4), str(1)
memory usage: 2.0 KB
None
Squeezed text (50 lines).
```

Scenario 2: Simple Visualization

Description: In this task we need to extract particular columns from the dataset, and plot the line graph for the same. While plotting we need to add the labels for both the axis and title to the chart. Once the chart is created we need to observe the trend whether it is increasing or decreasing.

Task	Description
1	Extract Year and Total columns.
2	Plot a line graph showing Total tracks over years.
3	Add Title and X and Y labels
4	Identify whether the trend is increasing or decreasing.

Task Workflow:

For the given set having multiple columns, if we need to print any particular columns we do achieve that by mentioning the column names as shown below.

```
#Extract Year and Total columns.
print(df[['Total','Year']])
```

This code will display only the columns with the names Total and Year as shown below.

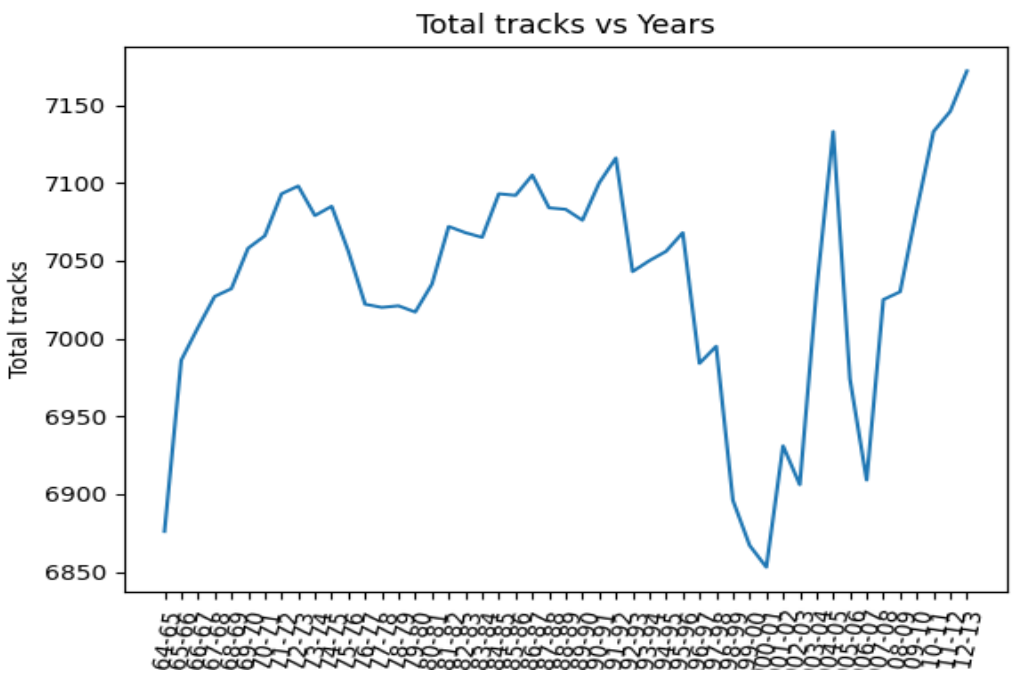
```
    Total      Year
0    6876   1964-65
1    6986   1965-66
2    7007   1966-67
3    7027   1967-68
```

There are 49 rows in the given dataset. It is a cropped image, and in the output we can see all the remaining data as expected.

In python we have matplotlib module which provides so many sub-modules like pyplot to get the graphical representation of the given data. We can display the required data in the form of 5 different types of graphs. As per the requirement we can use plot() function to display line graph of two particular columns as shown below.

```
#Plot a line graph showing Total tracks over years.
#Add: Title X and Y labels
#Identify whether the trend is increasing or decreasing.
plt.plot(df['Year'],df['Total'])
plt.xticks(rotation=80)
plt.xlabel('Year')
plt.ylabel('Total tracks')
plt.title('Total tracks vs Years')
plt.savefig('Graphs/total_tracks_over_year.png')
plt.show()
```

In the above code we are passing df['Years'] to X-axis and df['Total'] to Y-axis. And xticks() is a function to rotate the values in X-axis slightly to the right (like Italic format). xlabel("Label_name") and ylabel(Label_name) are the functions to add labels to the x and y axis respectively. title("Title") function is to add title to the graph and savefig("path") function is used to save the graph in a specified location/path. The graphical representation of the above code is as follows.



Conclusion: The graph is showing lot of variations like for some years there are less tracks installed and for some years we can see highest tracks installed. According the data, we can say the trend kept on increasing after the year 2006-07.

Scenario 3: Filtering + Bar Chart

Description: In this task we need to filter out the data from a specific year 2000. And by considering only gauge data we need to plot grouped bar chart for comparing the all three gauges. While plotting the chart we need to add suitable title and labels for both the axis and identify which gauge dominates in recent years

Task	Description
1	Filter the dataset for years after 2000.
2	Select Broad Gauge, Metre Gauge, and Narrow Gauge.
3	Plot a grouped bar chart comparing all three gauges.
4	Add legend and proper labels.
5	Identify which gauge dominates in recent years.

Task Workflow:

The requirement is to filter the data after the year 2000. To achieve that we need to convert the dataset values to numeric. In pandas we have a function called pd.to_numeric(). This function accepts columns and change those column values into numeric type. If we have any blank spaces or –(hyphen) or hidden strings in the cells, program may crash. To avoid that we can use errors='coerce'.

And also we astype() function, which is used to convert any type of data to required type. In the given dataset we have strings in the column Year. And as per the requirement we need to filter out the data after the 2000. As the dataset containing year in the format 2000-01, it will be considered as string. Not an integer. To achieve the task, we can use slicing method to take the first 4 digits from the data and compare with the given value. Once we have the data after comparison we can store it in a new variable and print it as shown below.

```
#3.1 Filter the dataset for years after 2000.
#Create a temporary filter without changing df
#This converts '2000-1' -> '2000' -> 2000 just for the comparison
filt = pd.to_numeric(df['Year'].astype(str).str[:4], errors='coerce') > 2000
print(df[filt])
```

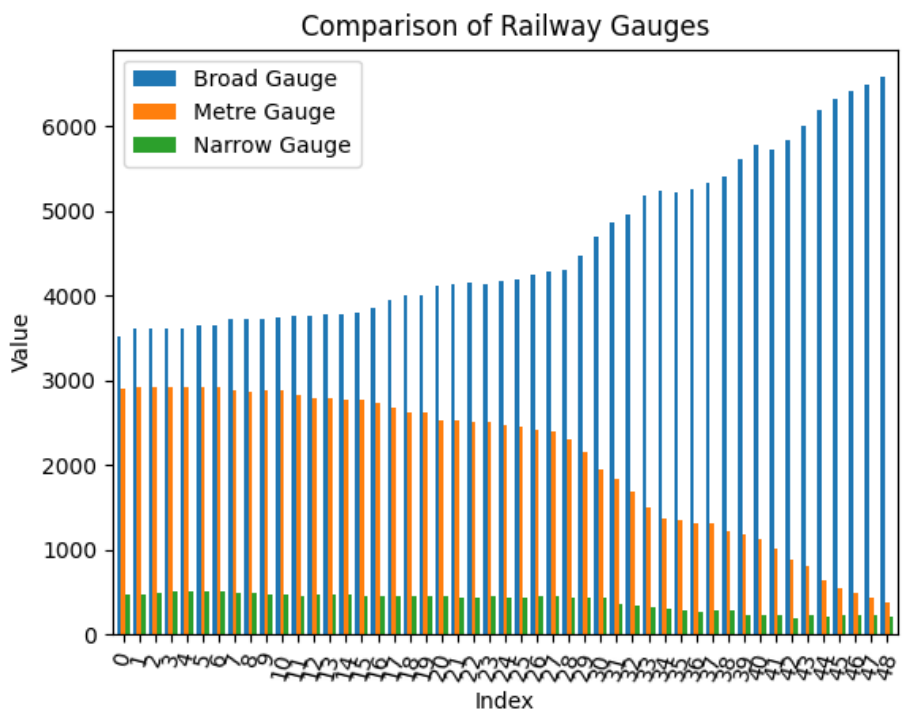
This code will give the data after the year 2000 as shown below.

	Year	Broad Gauge	Metre Gauge	Narrow Gauge	Total
37	2001-02	5338	1310	283	6931
38	2002-03	5402	1220	284	6906
39	2003-04	5611	1190	230	7031
40	2004-05	5780	1134	219	7133
41	2005-06	5727	1018	229	6974
42	2006-07	5830	888	191	6909
43	2007-08	6000	806	219	7025
44	2008-09	6185	642	203	7030
45	2009-10	6320	543	220	7083
46	2010-11	6419	494	220	7133
47	2011-12	6494	432	220	7146
48	2012-13	6577	379	216	7172

The next step is to plot the bar chart for all the gauge values in the provided dataset. To achieve that we can use pyplot module and plot() function as mentioned earlier. So the code to plot the graph and and save it in a folder is as follows.

```
#Select Broad Gauge, Metre Gauge, and Narrow Gauge.
#Plot a grouped bar chart comparing all three gauges.
df[['Broad Gauge', 'Metre Gauge', 'Narrow Gauge']].plot(kind='bar', width=0.8, rot=75)
#Add legend and proper labels.
plt.title("Comparison of Railway Gauges")
plt.ylabel("Value")
plt.xlabel("Index")
plt.legend()
#Identify which gauge dominates in recent years.
plt.savefig('Graphs/Broad_Metre_Narrow_bar.png')
plt.show()
```

This code will create a bar chart and saves the graph in the mentioned path. And the output is as follows.



Conclusion: From the graph we can say that Broad Gauge dominates in recent years.

Scenario 4: Feature Engineering + Pie Chart

Description: In this task we need to calculate the sum of all the gauges individually and create a new DataFrame/Series for those values. Once the structure is created we need to create a pie chart to display the contribution of each gauge in percentage. And provide the data which gauge contribute the most.

Task	Description
1	Calculate total sum of each gauge across all years.
2	Create a new structure (Series/DataFrame) for totals.
3	Plot a pie chart showing percentage contribution.
4	Add percentage labels (autopct).
5	Interpret which gauge contributes the most.

Task Workflow:

To analyze the contribution of each gauge type, we need to calculate the total/sum of each gauge of every year. To achieve that we can use sum() function which will calculate the total/sum of all the values in a given range. As we have multiple columns we can create a separate variable for each column to store the sum of those columns as shown below.

```
#Calculate total sum of each gauge across all years.
broad_sum=sum(df['Broad Gauge'])
metre_sum=sum(df['Metre Gauge'])
narrow_sum=sum(df['Narrow Gauge'])
print(broad_sum,metre_sum,narrow_sum)
```

This code will print the sum of individual columns. Now to create a new pandas Series we can use pd.Series() constructor function by passing the newly created variable values to it. This function will accept lists, numpy arrays, dictioneries as argument. So we are passing the variable along with keyvalues as shown below.

```
#Create a new structure (Series/DataFrame) for totals.
new_df=pd.Series({'Broad Gauge':broad_sum,
                  'Metre Gauge':metre_sum,
                  'Narrow Gauge':narrow_sum})
print(new_df)
```

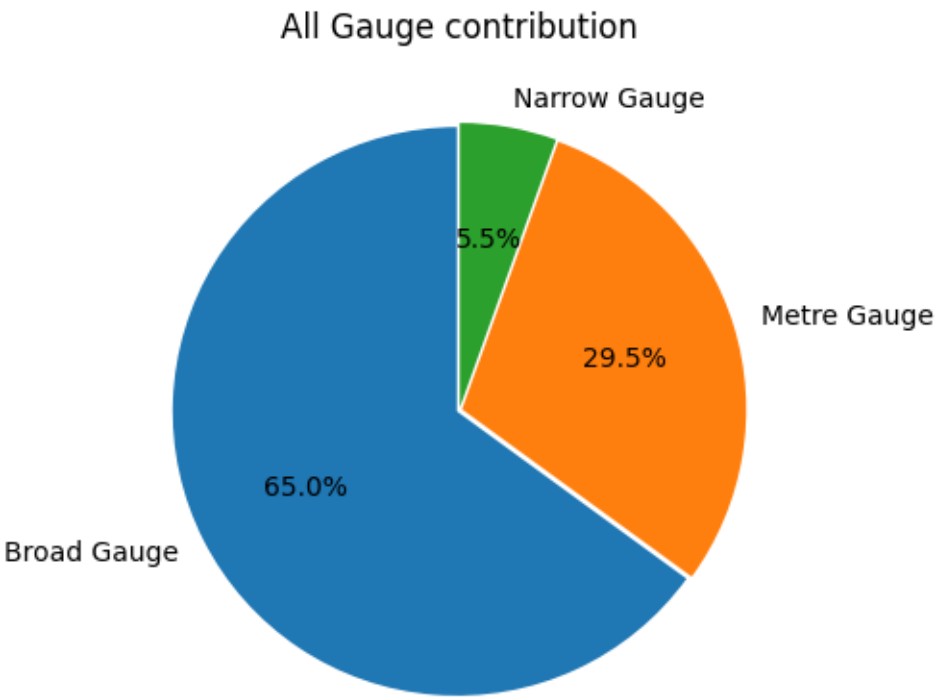
This will create a new pandas Series with the sum of gauges with their total as shown below.

```
Broad Gauge     224066
Metre Gauge     101798
Narrow Gauge     18953
dtype: int64
```

To represent the contribution of each gauges in pictorial form, we can use pie chart for better understanding. plt.pie() function is used to create pie chart. It accepts the values for X-axis and labels for Y-axis. autopct is a parameter of plt.pie() function to display percentage share of each slice directly on pie chart. explode parameter is used to provide space between each slices in the pie chart. And startangle parameter is to start the pie chart slicing from 90 degrees.

```
#Plot a pie chart showing percentage contribution.
#Add percentage labels (autopct).
plt.clf() # Clear Figure-It is a function used to wipe the current plotting window clean.
plt.pie(new_df.values,labels=new_df.index,autopct='%1.1f%%',explode=(0.01,0.01,0.01),startangle=90)
plt.title('All Gauge contribution')
plt.savefig('Graphs/All_Gauge_contribution.png')
plt.show()
```

The graph after running the above code is as follows.



Conclusion: From the pie chart we can easily say that Broad Gauge have the most contribution for the tracks with 65.0%.

Scenario 5: Advanced Analysis + Multiple Graphs (Hard)

Description:

The requirement is to create new gauge columns with the percentage values of each gauge. After creating the columns with percentage calculate the yearly growth of total tracks by using nympy.diff() function. After that provide the graphical representation of each gauges using line and stacked bar

charts. And also findout the year with highest growth and is there any declines in any gauges. Finally provide the conclusion that which gauge is growing rapidly.

Task	Description
1.	**Create new columns:** • % Broad Gauge • % Metre Gauge • % Narrow Gauge
2.	Use NumPy (**np.diff**) to calculate yearly growth of Total tracks.
3.	**Plot**: • Line graph for all gauges • Stacked bar chart showing composition
4.	**Highlight**: • Years with highest growth • Decline in any gauge
5.	Provide a **final conclusion**: ☞ "Is the railway system shifting towards a single dominant gauge?

As per the requirement in task1, we can create the columns by using the same variable/DataFrame name df and passing the column names as arguments and calculate the percentage of each gauge as shown below.

```
# 5.1 Create new columns: % Broad Gauge % Metre Gauge % Narrow Gauge
df['% Broad Gauge'] = (df['Broad Gauge'] / df['Total']) * 100
df['% Metre Gauge'] = (df['Metre Gauge'] / df['Total']) * 100
df['% Narrow Gauge'] = (df['Narrow Gauge'] / df['Total']) * 100
print(df)
```

The above code will create 3 columns by the name % ------ Gauge and each row will have the percentage value from the columns from dataset as shown below. There are 49 rows in the given dataset. It is a cropped image, and in the output we can see all the remaining data as expected.

```
       Year  Broad Gauge  ...  % Metre Gauge  % Narrow Gauge
0   1964-65         3521  ...      42.044793        6.748109
1   1965-66         3603  ...      41.697681        6.727741
2   1966-67         3615  ...      41.544170        6.864564
3   1967-68         3601  ...      41.554006        7.200797
4   1968-69         3603  ...      41.538680        7.224118
```

For the next task we need to use numpy.diff() function to calculate the difference between consecutive elements in an array. In the below code we are calculating difference from the column Total using np.diff() and saving it in a variable. After that we are creating a new column (not affecting original dataset). While inserting values to that column we are passing (0,0). We insert a 0 at the start to match the dataframe's row count.

```
# 5.2 Yearly Growth of Total Tracks using np.diff
# We insert a 0 at the start to match the dataframe's row count
growth_values = np.diff(df['Total'])
df['Yearly Growth'] = np.insert(growth_values, 0, 0)
print(df['Yearly Growth'])
```

The above code provide the result as follows.

```
0        0
1      110
2       21
3       20
4        5
```

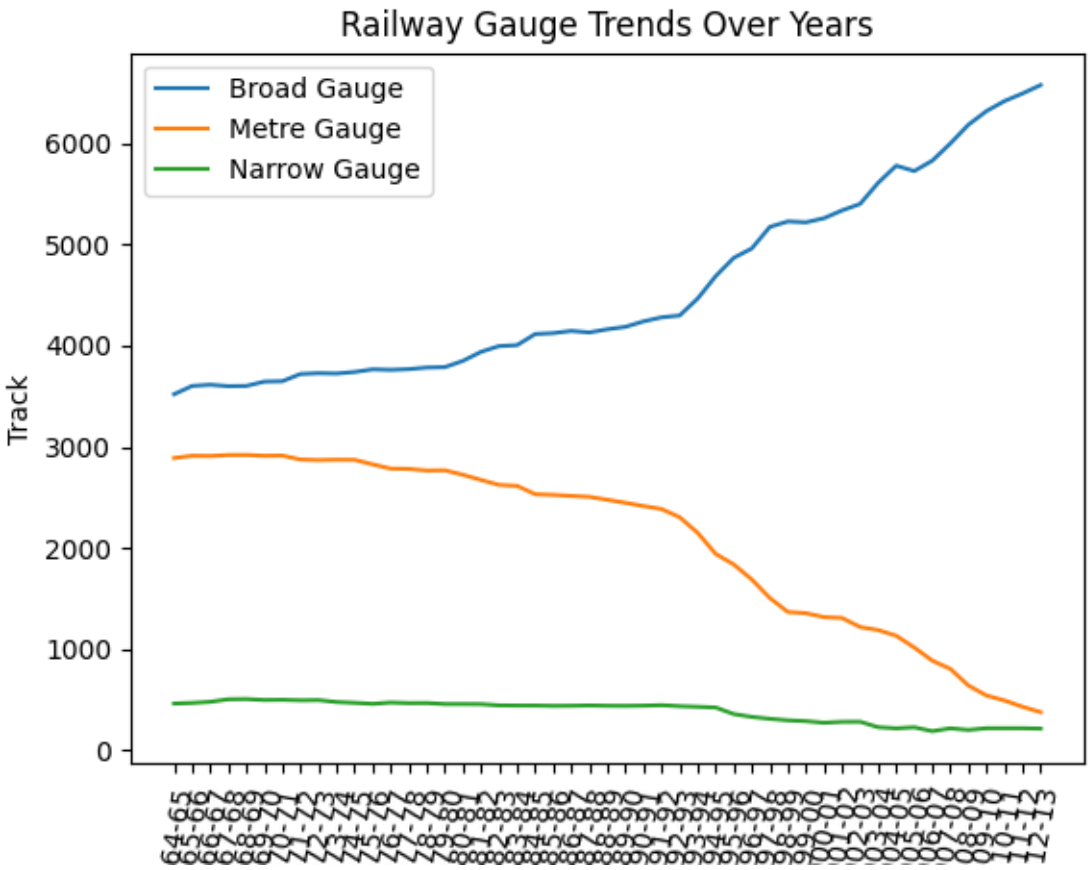
For the next task we need to create graphical representation of the gauges using Line graph and Stacked bar chart showing composition.

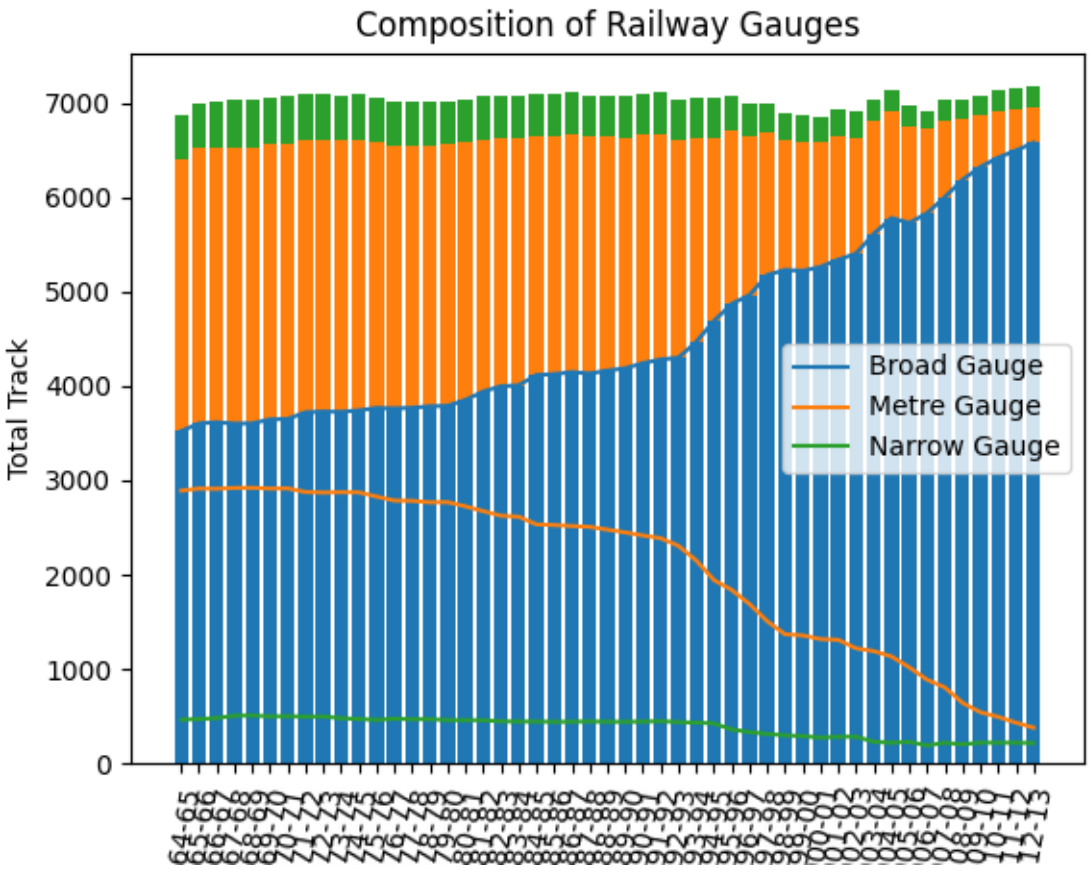
```
# 5.3 Plot: ○ Line graph for all gauges and ○ Stacked bar chart showing composition
for gauge in ['Broad Gauge', 'Metre Gauge', 'Narrow Gauge']:
    plt.plot(df['Year'], df[gauge], marker='o', label=gauge)

plt.title('Railway Gauge Trends Over Years')
plt.xlabel('Year')
plt.ylabel('Track')
plt.xticks(rotation=80)
plt.legend()
plt.savefig('Graphs/Railway_Gauge_Trends_Over_Years.png')

# Stack the bars using the 'bottom' parameter
# Force the 'Year' column to be strings (helps prevent numeric scaling)
df['Year'] = df['Year'].astype(str)
# Plot the bars
plt.bar(df['Year'], df['Broad Gauge'], label='Broad Gauge')
plt.bar(df['Year'], df['Metre Gauge'], bottom=df['Broad Gauge'], label='Metre Gauge')
plt.bar(df['Year'], df['Narrow Gauge'], bottom=df['Broad Gauge'] + df['Metre Gauge'], label='Narrow Gauge')

# Manually set the ticks and labels
plt.xticks(ticks=range(len(df['Year'])), labels=df['Year'])

plt.title('Composition of Railway Gauges')
plt.xlabel('Year')
plt.ylabel('Total Track')
plt.xticks(rotation=80)
plt.savefig('Graphs/Composition_of_Railway_Gauges.png')
```

In the above code we are using pyplot sub-module functions like plot(), title(), bar(), xlabel(), ylabel(), xticks() and savefig().

In the stacked bar chart by default it will display index values on X-axis. To display Year on X-axis we can manually set the ticks as shown in the code.

The graphs line graph and stacked bar chart after executing the above code are as follows.


Railway Gauge Trends Over Years
Broad Gauge
Metre Gauge
Narrow Gauge
Track
0
1000
2000
3000
4000
5000
6000


Composition of Railway Gauges
Broad Gauge
Metre Gauge
Narrow Gauge
Total Track
0
1000
2000
3000
4000
5000
6000
7000

In the next task we need to check for the Years with highest growth and Decline in any gauge. To achieve that we can use max() and idxmax() functions to display highest growth value and location of that value, which is year respectively as shown below. And to check declines in any gauge we are using for loop to access each column values one by one. After that checking each value with the condition if difference is less than 0(zero) or not. If the difference is less than 0(zero) we can append those values to a list and display it as shown below.

```
# Decline check
declines = []
for gauge in ['Broad Gauge', 'Metre Gauge', 'Narrow Gauge']:
    diffs = np.diff(df[gauge])
    if any(diffs < 0):
        declines.append(gauge)

print(f"Highest Growth: {highest_growth_year} with {highest_growth_val} km")
print(f"Gauges with decline: {', '.join(declines)}")
```

Conclusion:

After carefully observing the graphs and outputs of the above code we can conclude that there is a rapid shift toward a single dominant gauge which is Broad Gauge.

Final Conclusion:

In this project we saw the record of how many gauges and types of gauges installed per year. With the help of pandas, numpy and matplotlib we can see different trends and which gauge is being used over the years.

What you have learned:

In this project we learned about Data Pre-processing, means how we can load the data to pandas DataFrame, how to view the data and perform operations on those data using different functions.

With the help Pandas we learnt how to handle the dataframe, access the datasets, converting the datatype to another datatype, replacing the Nan with other values etc..

By using NumPy Calculations we learnt that how to calculate difference, how to insert the values to a column.

As a part of Visualization we learnt how to plot the graphs like Line graph, Bar chart, Pie chart and Stacked chart as per the requirement.